

Received 5 March 2026, accepted 18 March 2026, date of publication 26 March 2026, date of current version 2 April 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3677890

RESEARCH ARTICLE

Multicore Implementation of Kyber on RP2350A for Resource Constrained Environments

ACHINTYA NIGAM^{ID}, AVANISH GHARAT, MEHUL BHUSHAN^{ID},
AND ISLABUDEEN MOHAMED MEERASHA^{ID}

School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu 632014, India

Corresponding author: Islabudeen Mohamed Meerasha (islabudeen.m@vit.ac.in)

This work was supported in part by Vellore Institute of Technology, Vellore for covering the publication costs of this research article.

ABSTRACT Quantum computers pose a serious threat to conventional public-key cryptography, motivating the adoption of post-quantum schemes such as Kyber. Resource-constrained devices, however, face challenges in sustaining the added computational cost. This paper implements and evaluates single-core and multicore variants of Kyber on the Raspberry Pi Pico 2 W (RP2350A), with emphasis on runtime and energy efficiency. Across parameter sets, multicore key generation reduces runtime by approximately 26-28%, while encapsulation and decapsulation exhibit more modest improvements of up to 3-4%, with minor variance across security levels and sampling sizes. Power measurements indicate that total energy per operation decreases by 8.9-10.9%, despite a $1.3\times$ increase in transient peak current, resulting in an overall energy-efficiency improvement of approximately 17% when normalized per unit time.

INDEX TERMS Post-quantum cryptography, CRYSTALS-Kyber, multicore microcontrollers, Raspberry Pi Pico 2 W, energy efficiency, IoT security, embedded cryptography, power consumption.

I. INTRODUCTION

Quantum computers have not yet advanced significantly enough to compromise the most commonly used cryptographic systems. Still, the rapid and sustained progress in quantum hardware development has raised serious concerns within the cryptographic research community.

Recent progress in quantum computing has reinforced the need for practical post-quantum cryptographic (PQC) deployments, particularly in resource-constrained environments. Rather than focusing on specific hardware milestones, this work is motivated by the broader requirement to make standardized post-quantum algorithms efficient and deployable on embedded platforms with limited computational and energy resources.

This study focuses on the Raspberry Pi Pico 2 (RP2350A) as a representative example of a compact, dual-core embedded platform.

The goal here is to assess how feasible it is to run the CRYSTALS-Kyber key encapsulation mechanism (KEM) on

such hardware. By taking advantage of the microcontroller's dual-core architecture, this research explores parallelization methods and scheduling strategies to reduce execution time and energy consumption while preserving accuracy and security.

Kyber was selected as the representative post-quantum KEM due to its NIST standardization [1], making it the de facto post PQC baseline for embedded deployments. Among lattice candidates, Kyber delivers the highest performance (cycles/bytes being the smallest on Cortex-M4/M33 as per PQM4 benchmarks), enabling realistic multicore scaling analysis on resource-constrained RP2350. Its wide adoption, used in Chrome 131+, Linux kernel 6.14 crypto, and AWS KMS, ensures relevance, while the representative lattice workload (NTT-heavy polynomial arithmetic, 384×384 matrix ops) stresses multicore SRAM bandwidth and synchronization, generalizing findings to other Module-LWE KEMs (Saber, NTRU-HRSS). Kyber512 results ($1.31\times$ speedup) thus benchmark multicore PQC viability rather than algorithm-specific optimization.

This work differs from prior embedded and multicore Kyber implementations in several key aspects. While

The associate editor coordinating the review of this manuscript and approving it for publication was Md. Abdur Razzaque^{ID}.

TABLE 1. Kyber runtime comparison across security levels and platforms (single-core vs. multicore where applicable).

Security Level	Platform	Core	Keygen (μ s)	Encap (μ s)	Decap (μ s)	Clock
Kyber512	RP2350A	M33	6041 $\rightarrow$ 4155	7064 $\rightarrow$ 6814	8683 $\rightarrow$ 8398	150MHz
Kyber512	STM32F4	M4	4600	3400	5900	168MHz
Kyber768	RP2350A	M33 $\times$ 2	9905 $\rightarrow$ 6899	11465 $\rightarrow$ 10567	13633 $\rightarrow$ 12695	150MHz
Kyber1024	RP2350A	M33 $\times$ 2	15408 $\rightarrow$ 10773	17265 $\rightarrow$ 16097	20038 $\rightarrow$ 8837	150MHz

previous studies have explored optimization on Cortex-M and ESP32 platforms, this study presents one of the first detailed multicore implementations of Kyber on the RP2350A based Raspberry Pi Pico 2 W, a newly released dual-core microcontroller with distinct architectural constraints. The proposed design introduces a lightweight message-box-based task-partitioning strategy tailored for low-memory environments and evaluates its impact not only on runtime but also on energy consumption and current behavior. Unlike existing work that focuses primarily on throughput, this study provides a combined performance-energy analysis and investigates the practical limits of coarse-grained parallelism for key generation, encapsulation, and decapsulation on a resource-constrained dual-core microcontroller unit (MCU).

The runtime results in Table 1 highlight the benefit of exploiting multicore parallelism on the RP2350A for Kyber. For Kyber512 on the Cortex-M33, moving from the baseline to the multicore/parallel variant reduces key generation from 6041 μ s to 4155 μ s, while encapsulation and decapsulation improve more modestly (7064 μ s to 6814 μ s and 8683 μ s to 8398 μ s, respectively), indicating that key generation contains a larger parallelizable fraction. Across higher security levels, similar trends are observed: Kyber768 and Kyber 1024 on the dual-core M33 $\times$ 2 show substantial reductions in key generation time (9905 μ s to 6899 μ s and 15408 μ s to 10773 μ s), with smaller but consistent gains for encapsulation and decapsulation. The table also provides context against prior Cortex-M4 results (STM32F4 at 168 MHz), underscoring that absolute timing depends on both micro architecture and clock frequency, but that within the same RP2350A platform, multicore execution yields clear latency reductions for Kyber KEM operations.

II. CONTRIBUTIONS

This work makes the following contributions:

- 1) Platform-specific deployment: Porting and adapting the CRYSTALS-Kyber reference implementation to the Raspberry Pi Pico2W (RP2350A), including integration with the on-chip TRNG. Multicore Implementation of Kyber through independent parallelization using the message box model.
- 2) Multicore execution model: Design and implementation of a lightweight message-box based task partitioning strategy for coarse-grained parallelization of Kyber operations on a dual-core resource-constrained microcontroller.

- 3) Performance-energy co-evaluation: Comprehensive experimental evaluation of single-core vs multicore execution across Kyber parameter sets, measuring runtime, current draw, and total energy consumption on real hardware.
- 4) Systems-level insights: Analysis of which Kyber stages benefit most from parallelism, showing significant gains in key generation and limited benefits in decapsulation due to sequential dependencies.

III. LITERATURE REVIEW

Kyber is a KEM that achieves Indistinguishable under Chosen Cipher-Text 2 (IND-CCA2) security, grounded in the computational hardness of the Module Learning With Errors (Module-LWE) problem.

The underlying public-key encryption (PKE) construction of Kyber, which is secure against chosen-plaintext attacks (CPA) bears conceptual and structural similarities to the NTRU-LPR scheme [2]. NTRU-LPR is itself a construction based on the Ring-LWE problem.

TABLE 2. A size comparison table for PQC algorithms at NIST security level 1 aggregated from the tables mentioned in "Evaluating the performance of post-quantum secure algorithms in the TLS protocol." [3].

Algorithm	Size (in bytes)		
	Ciphertext	Public Key	Secret Key
Kyber512	768	800	1632
ntruhs2048509	699	699	935
lightSaber	736	672	1568

TABLE 3. A mean time comparison table on i7-6700k @4GHZ CPU for PQC algorithms at NIST security level 1 aggregated from the tables mentioned in "Evaluating the performance of post-quantum secure algorithms in the TLS protocol." [3].

Algorithm	Mean Time (in ms) on i7-6700k @4 GHz		
	KeyGen	Encaps	Decaps
Kyber512	0.065	0.088	0.112
ntruhs2048509	2.965	0.181	0.469
lightSaber	0.028	0.033	0.035

A comprehensive performance evaluation was conducted by independent researchers to assess the efficiency of post quantum public-key algorithms employed for key exchange within the Transport Layer Security (TLS) protocol, with a particular emphasis on the finalist algorithms from NIST's third round of PQC standardization.

TABLE 4. A mean time comparison table on i5-8250U @1.6GHZ CPU for running PQC algorithms at NIST security level 1 aggregated from the tables mentioned in "Evaluating the performance of post-quantum secure algorithms in the TLS protocol" [3].

Algorithm	Mean Time (in ms) on i5-8250U @1.6 GHz		
	KeyGen	Encaps	Decaps
Kyber512	0.094	0.128	0.159
ntruhs2048509	4.287	0.264	0.691
lightSaber	0.042	0.049	0.052

Valuable insights were provided by this study into the comparative performance and the practical viability of the candidate schemes under realistic network and computational conditions. The experimental results highlighted notable variations in performance among the contenders, underscoring the importance of balancing theoretical security guarantees with real-world efficiency.

Among all evaluated schemes, CRYSTALS-Kyber emerged as the most efficient and implementation-friendly KEM, a finding that strongly validated NIST's decision to select it as the primary post-quantum key exchange standard.

With this combination of high-speed key generation, encapsulation, and decapsulation across the CPU benchmarks in Tables 3 and 4, Kyber512 shows consistently low latency (0.065/0.088/0.112 ms (KeyGen/Encaps/Decaps) on the i7-6700k and 0.094/0.128/0.159 ms on the i5-8250U) while maintaining moderate parameter sizes (768 B ciphertext, 800 B public key, 1632 B secret key) as summarized in Table 2. Although lightSaber is faster in these measurements, Kyber512 remains within sub-millisecond operation times and avoids the very large key-generation overhead observed for ntruhs2048509, indicating that Kyber offers a practical balance between computational cost and bandwidth footprint for TLS-style handshakes.

Conversely, NTRU variants despite their similar underlying lattice-based hardness assumptions, had relatively poor performance. This was specially observed in the key generation and encapsulation latency.

Although the key and ciphertext sizes of NTRU were similar to the size of other lattice-based KEMs like Kyber and Saber, NTRU implementations were more prone to incur higher computational costs and less predictable scaling on resource-constrained hardware, especially on resource-constrained hardware, compared to other lattice-based KEMs [3].

This performance difference, although not disqualifying, made NTRU a less desirable choice where high throughput or latency-sensitive behavior is required, such as secure web communications and embedded systems.

Another strong member of the structured lattice family, Saber algorithm, was also shown to be slightly more efficient in specific operations, such as the use of memory and easy implementation.

Nonetheless, in the assessment conducted by NIST, it was established that incorporating several structured lattice-based KEMs in the final standard set would offer a small amount of

practical diversification as regards both security assumptions and algebraic structure. That is to say, Kyber and Saber had comparable design principles based on the Module-LWE problem, and their combination would not add much additional resilience to the post-quantum cryptographic ecosystem.

Although both Saber and Kyber proved to be safe and resistant to classical and quantum attacks, Kyber became the ultimate standard.

The reason behind making this selection was not only Kyber having better runtime performance compared to other algorithms except Saber, but also the maturity of its security analysis, the clarity and transparency of its parameter selection, as well as the extent of independent cryptanalytic scrutiny that it had received by that point in the standardization process [4].

All these factors gave NIST and the cryptography community in general greater assurance that Kyber was highly secure and deployable in the long term. In Contrast, as much as Saber was doing better than Kyber, their security was not examined as much as it was in the case of Kyber.

The resulting choice of Kyber is therefore a considered and informed decision; one that offers a high level of theoretical certainty, a pragmatic approach of implementation, and a system-level performance.

Its use as a post-quantum standard in the key exchange component of TLS not only marks a significant step in the process of quantum-resistant cryptography becoming standard but also establishes precedent on how future PQC schemes will need to balance efficiency, transparency, and security assurance as the computational paradigm evolves.

A. PERFORMANCE ON EMBEDDED PLATFORMS

One of the most susceptible parts of the present-day cybersecurity architecture is IoT and embedded system platforms. The main reason behind this weakness is the hardware constraints such as limited processing capability, memory space, and energy supply, which are inherent to them.

These limitations might require the use of reduced-bit security functions or watered-down cryptographic designs, which might not offer sufficient security against current or future quantum-capable attackers.

Due to the exponentially growing number of IoT devices over the last ten years, the necessity to assess the feasibility and effectiveness of post-quantum cryptographic algorithms in such resource-limited conditions has increasingly been felt [5], [6].

PQC algorithms themselves are computationally intensive, and their practicality on resource-limited systems has to be investigated before they can be implemented effectively in them.

An attempt to investigate the NTRU cryptosystem as an IoT endpoint encryption system was carried out through a feasibility study [7]; the study indicated that NTRU could be optimally implemented in low-power microcontrollers. The

results have put emphasis on the good performance attributes of NTRU, such as the rapid key generation and encapsulation, and the relatively small memory footprint compared to other post-quantum protocols.

These features ought to render NTRU especially effective in embedded systems where energy effectiveness and real-time reactivity is crucial. The research thus cited NTRU as the potential alternative or supplement of traditional public-key cryptographic algorithms in the resource-constrained IoT system to provide long-term data confidentiality, integrity and authentication in the face of quantum threats.

Nevertheless, the research findings were also discussed in the same study with a number of limitations limiting the generalizability and relevance to modern circumstances.

It is worth noting that it used old parameter sets, which might not be consistent with the latest recommendations of the NIST PQC process, and did not directly protect against chosen ciphertext attacks (CCA) which is a very important requirement to assure real-world resilience.

The experiments presented in the present paper were performed on older microprocessor architectures, namely on the ARM Cortex-M0 family, and this is not indicative of the possibilities of newer embedded systems with more advanced cryptographic acceleration and hardware security modules.

Although the findings proved the viability, it did not give a thorough evaluation of the performance of NTRU on the current hardware platforms.

To fill these gaps, one can assume that analogous experiments may be replicated with newer architectures, such as the ARM Cortex-M33, which is the processor used in a Raspberry Pi Pico 2, and the processors with the RISC-V instruction set architecture.

These platforms have better computational capability as well as have emerging security extensions like hardware-based random number generators and the constant time arithmetic operations that are useful in countering side-channel attacks.

Another contribution of the work was the creation of a time-independent (constant-time) implementation of NTRU, in particular tailored to resist timing attacks, which is a widespread and critical threat to embedded cryptosystems.

This method prevents the enemies of the encryption and decryption process from making inferences about the secret information by monitoring the performance of the encryption and decryption process within a specific time frame, irrespective of the source data. Such implementation-level protections have been added due to the increasing awareness that algorithmic security needs to be augmented with side-channel resistance to provide overall protection in practice.

Although the results of the experiment were encouraging, the researchers noted that it is still difficult to make a straightforward performance comparison between NTRU and other PQC schemes, which include Kyber or Saber.

This difficulty stems from differences in experimental configurations, including hardware specifications, compiler

optimizations, and benchmarking methodologies, which can significantly influence reported results.

Establishing standardized evaluation frameworks for PQC algorithms on embedded systems is thus an important next step for the research community and this would surely enable fair and reproducible assessments of their efficiency and security.

The findings from this feasibility study do definitely provide strong empirical evidence supporting the viability of NTRU as a lightweight and secure lattice-based cryptosystem for low-power embedded and IoT devices.

Its ability to achieve efficient key exchange and encryption while maintaining a modest memory footprint makes it a very attractive option for future-proofing the IoT ecosystem against quantum-enabled adversaries.

As newer microcontroller platforms continue to evolve, and as PQC standardization efforts mature, it is anticipated that optimized and side-channel-hardened implementations of NTRU and similar schemes will play a central character in the upcoming generation of quantum-resistant embedded security architectures.

TABLE 5. Comparison of NTRU + Contiki NG and bare-metal versions on ARM Cortex-M4. [8].

Algorithm	Clock Cycles		
	KeyGen	Encaps	Decaps
Ntruhs2048509 + Contiki-NG (-Os)	136034849	5371816	9659199
Ntruhs2048509 (clean)	107331110	2876597	7495670
Kyber512 (clean)	649678	884848	985258
Lightsaber (ref)	1051530	1538646	1861934

Another study evaluated “The performance of the NTRU cryptosystem on resource-constrained IoT edge devices” [8], providing critical insights into the feasibility of lattice-based post-quantum schemes in real-world, low-power environments.

In this investigation, the authors implemented NTRU on the Contiki-NG operating system, an open-source platform widely used in wireless sensor networks and IoT deployments.

Evident from Table 5, on an ARM Cortex-M4, Ntruhs2048509 under Contiki-NG(-Os) requires 36,034,849 cycles for key generation, 5,371,816 for encapsulation, and 9,659,199 for decapsulation, whereas the bare-metal clean implementation reduces these to 107,331,110, 2,876,597, and 7,495,670 cycles, respectively (-21.1% for KeyGen, -46.5% for Encaps, and -22.4% for Decaps). In contrast, Kyber512 (clean) is substantially lighter at 649,678/884,848/985,258 cycles (KeyGen/Encaps/Decaps), while Lightsaber (ref) reports 1,051,530/1,538,646/1,861,934 cycles. Relative to Lightsaber, Kyber512 is 38.2% faster for KeyGen, 42.5% faster for Encaps, and 47.1% faster for Decaps; relative to clean NTRU, Kyber512 uses about $165\times$ fewer cycles for KeyGen, $3.25\times$ fewer for Encaps, and $7.61\times$ fewer for Decapsulation. CRYSTALS-Kyber and Saber were always

faster than NTRU both in their unoptimized (clean) code, as well as when instantiated in the Contiki-NG operating system, as shown in Table 5.

A weakness that was found was that NTRU generated keys slower than the competing post-quantum schemes, and, therefore, was not well suited to time-sensitive applications like session key negotiation in IoT networks.

To add to these findings, another study has performed a broad-based performance analysis of different post-quantum cryptographic designs which highlights the importance of hardware acceleration to achieve practical efficiency in small-sized equipment [9].

The experiments indicated that implementation with hardware acceleration can significantly decrease computation time, energy consumption, and latency on a set of PQC algorithms. Kyber had very noteworthy results.

The Kyber key establishment functions were again always well performing in all the experimented platforms and the maximum time taken to execute any operation at any level of security was merely 17 milliseconds. Such a high level of efficiency suggests that Kyber can be deployed in constrained IoT and embedded systems as well, and can generate a secure key in a relatively short period of time without overwhelming the processor and consuming too much device power.

The performance and the energy consumption of Kyber and other post-quantum algorithms were also evaluated in studies that the same authors analyzed in their literature review.

One of the most interesting discoveries was that the Kyber (key exchange) and Dilithium (digital signatures) combination had a potential of requiring less energy than the classical algorithms like Elliptic Curve Digital Signature Algorithm (ECDSA) and Elliptic Curve Diffie-Hellman (ECDH). This finding implies that properly optimized lattice-based PQC primitives may provide quantum as well as energy efficiency equal to or beyond that of their classical analogues.

Nevertheless, the authors have also warned that most existing software-only realizations of post-quantum algorithms are computationally intensive, especially when using small and very-low-power embedded processors.

In order to deal with this, they suggested that one way forward could be to apply PQC schemes to Field-Programmable Gate Arrays (FPGAs) and eventually to Application-Specific Integrated Circuits (ASICs).

Hardware-based acceleration of this kind would allow even the most restricted devices to have strong cryptographic security. This would make quantum resiliency not costly in terms of energy efficiency and responsiveness. The designs with this type of help provided by hardware may become central in the development of quantum-safe embedded systems and, by extension, enable IoT networks to grow safely with the development of quantum computing.

In line with these findings, study that evaluated on the performance of post-quantum cryptographic secure algorithms in the TLS protocol was done [3].

It was additionally established that NTRU variants were not as good as Saber and CRYSTALS-Kyber even with similar key and ciphertext size consisting in the figure 1. black.

The performance of all the assessed cryptosystems was competitive, and frequently in the same range as or better than traditional elliptic curve public-key algorithms. What is more important, the performance of Kyber was similar to that of classical implementations in use in TLS, and its degradation was minimal when subjected to different network conditions, once again proving its appropriateness to real-world use.

A research gap that was identified in this study was that it failed to provide detailed measurements of the energy consumption. This especially applies to battery-constrained IoT and edge devices.

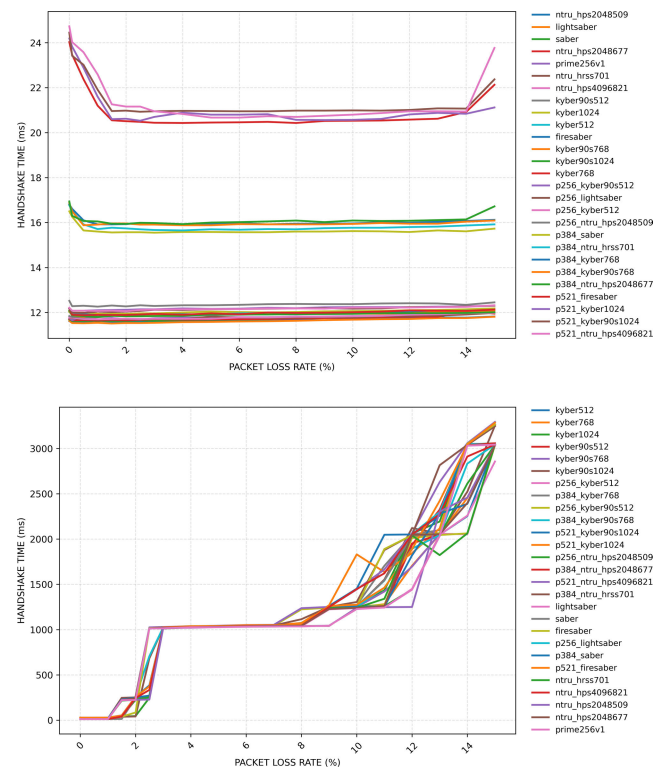


FIGURE 1. This figure compares the TLS handshake completion time for post-quantum (PQC) hybrid ciphers (Level 5 NIST security) against the standard P-256 elliptic curve Diffie-Hellman (ECDH) method. The First plot shows the median times, and the second shows the 95th percentile times, both measured under optimal Round-Trip Time(RTT) conditions. [3].

Future studies filling the gap can help get a more detailed picture regarding trade-offs between the level of energy efficiency, the security level and the computational speed. This will play a crucial role in scaling the use of PQC in heterogeneous embedded settings.

Another study examined the performance, energy consumption, and computational costs of post-quantum cryptographic (PQC) algorithms on embedded devices [10]. The authors specifically tested Kyber, Saber, and Classic McEliece for key exchange, and Dilithium, Falcon, and SPHINCS+ for digital signatures, using a Raspberry Pi Pico W. They also evaluated traditional algorithms such as

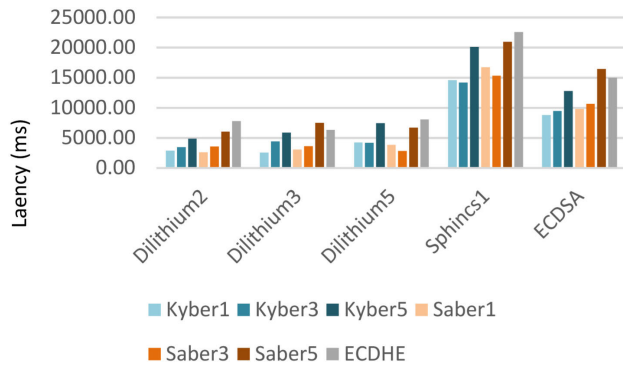


FIGURE 2. “TLS Handshake Latency on the Raspberry Pico W (Client)” [10].

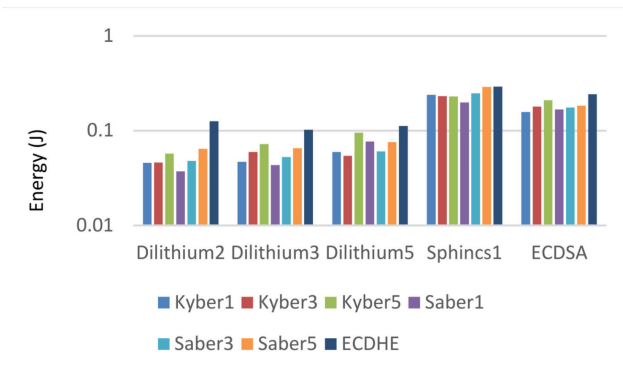


FIGURE 3. “Energy dissipation for a TLS handshake of a raspberry Pi pico W client.” [10].

ECDSA and ECDH, along with various PQC combinations, across a full TLS handshake.

Evident from Figure 1, under optimal RTT conditions, the median TLS handshake time remains relatively stable across packet loss rates for all schemes. For the PQC hybrid ciphers (Kyber-1024 and P-521 hybrids), median completion times decrease slightly from approximately 24-25 ms at 0% loss to about 20-21 ms at 1-2% loss, after which they remain nearly constant up to 15% loss, with only a small increase at the highest loss rate (approx 22-24 ms). In contrast, classical P-256 and NTRU/firesaber hybrids maintain lower median times of approximately 11.5-12 ms, exhibiting minimal variation (<0.5 ms) across the full loss range.

The 95th percentile handshake times show substantially greater sensitivity to packet loss. Up to 2% loss, all schemes remain below approximately 200 ms. However, beyond a 3% loss, completion times increase sharply to around 1,000 ms and continue to rise with higher loss rates. At 15% packet loss, 95th-percentile times are approximately 3,000-3,300 ms for PQC hybrids and around 3,000 ms for classical and hybrid alternatives. While median performance remains stable, worst-case latency degrades significantly under elevated packet loss, with PQC hybrids exhibiting slightly higher tail latency at the highest loss rates.

Figures 2 and 3 show that a combination of Kyber at NIST Security Level 1 and Dilithium at Level 2 was the most efficient in terms of performance. SPHINCS+ exhibited

the highest latency, followed by ECDSA, while ECDHE generally showed higher latency compared to others. Importantly, this work demonstrated that modern PQC algorithms can indeed operate effectively on highly resource-constrained embedded networked devices, challenging the assumptions made in earlier research that such schemes would be too demanding for low-power platforms. They, however, had not included NTRU in their study.

Table 6 summarizes representative embedded implementations of Kyber-512 key generation across several micro controller platforms and optimization approaches, using the PQM4 reference implementation on a Cortex-M4 as the base line. Prior work shows that substantial gains are achievable on single-core Cortex-M systems through architecture-aware tuning (e.g., DSP intrinsics and NTT-specific optimizations as in wolfSSL), while multicore platforms such as the dual core ESP32 can obtain more modest speedups by combining task parallelism with platform accelerators. The table also includes an ARMv8-M/Cortex-M33 class example that reports larger improvements via vectorization/SIMD-style software optimizations without multicore execution. In contrast, this work targets the RP2350A in the Raspberry Pi Pico 2 W and demonstrates that lightweight dual-core task-level parallelization (focused on NTT-heavy stages) yields a $1.31\times$ reduction in Kyber-512 key generation latency ($6041 \rightarrow 4155 \mu s$) at 150 MHz, highlighting the practical benefits of multicore parallelism for PQC on resource-constrained microcontrollers.

B. OPTIMIZATION TECHNIQUES

A study proposed a comprehensive methodology aimed at significantly reducing the development time for most post-quantum cryptographic (PQC) candidates, while simultaneously maintaining the integrity and accuracy of performance comparisons and candidate evaluations [15].

The proposed approach is centered on a hardware/software (HW/SW) co-design framework, which allows for an efficient partitioning of computationally intensive cryptographic operations between hardware accelerators and software components. The framework can be further enhanced through the optional integration of High-Level Synthesis (HLS) techniques, thereby enabling the automatic generation of hardware designs from high-level algorithmic descriptions. This strategy not only expedites the prototyping phase but also allows for rapid design-space exploration and architectural optimization.

To demonstrate the practicality of this methodology, the authors used NTRUEncrypt as a case study and implemented it on Xilinx Zynq FPGA devices.

The experimental results were impressive, encryption process achieving a $3.9\times$ speed-up and with the decryption process achieving an even greater $6.8\times$ speed-up compared to the pure software implementation.

The authors attributed the disproportionate improvement in decryption performance to the fact that a larger fraction of

TABLE 6. Comparison of embedded Kyber implementations and optimizations (Kyber-512 key generation).

Work	Platform	MCU	Frequency	Library	Optimization Strategy	Multicore	Speedup (Kyber-512 KeyGen)
PQM4 (Ref. impl.) [11]	STM32L4	Cortex-M4	120 MHz	PQClean	Clean C (no architecture-specific optimizations)	No	Baseline (~595k cycles)
wolfSSL [12]	STM32F4	Cortex-M4	168 MHz	Custom	DSP intrinsics + NTT tuning	No	~1.7× vs baseline
ESP32 Dual-core [13]	ESP32	Xtensa LX6 ×2	240 MHz	PQ-Crystals reference	Hardware accel + task parallelism	Yes	~1.21× vs single-core
ARMv8 optimized [14]	ARMv8-M	Cortex-M33 (est.)	~1500 MHz	Custom	Vectorization / SIMD optimization	No	~2.5× vs baseline
This work	Pico 2 W	RP2350A	150 MHz	PQ-Crystals reference	Dual-core task-level NTT parallelization	Yes	1.31× vs baseline

time-consuming operations within the decryption phase, such as polynomial multiplications and modular reductions, had been successfully offloaded to specialized hardware modules. This demonstrated the potential of the HW/SW co-design methodology to significantly accelerate PQC operations, particularly for schemes with heavy arithmetic workloads.

However, a subsequent study challenged the limitations of such hybrid approaches by presenting a fully hardware-based implementation of the Kyber algorithm on an FPGA [16].

In this work, the authors argued that the use of dedicated hardware for all computational components could eliminate software bottlenecks altogether, thereby achieving superior performance.

Their implementation results strongly supported this claim. The pure hardware design of Kyber achieved an execution time of only 30.5 μ s, vastly outperforming the 475 μ s observed in the HLS-based hybrid design, both measured at the Kyber parameter set $k = 2$.

This comparison demonstrates the profound performance gap between hybrid and fully hardware-based implementations, particularly in latency-sensitive environments where real-time encryption and decryption are critical.

The NIST Round 3 Algorithm Specifications and Supporting Documentation for Kyber [17] provide further insight into Kyber's internal structure, particularly its reliance on the Number-Theoretic Transform (NTT) for efficient polynomial arithmetic.

The NTT is employed to accelerate multiplication within the polynomial ring R_q , offering both high computational efficiency and a small memory footprint. According to the specifications, the NTT is designed to be extremely fast, consuming minimal resources while maintaining deterministic behavior which is an essential property for cryptographic consistency.

The specification also carefully constrains where the NTT can be applied within the algorithm. Its use is mandated only during the sampling of the public matrix A and the generation of the public key.

As a result, NTT-based multiplication is included in Kyber.CPAPKE which is the CPA-secure public-key encryption component of Kyber since the public matrix A is generated and stored in the NTT domain.

Similarly, multiplications involving the public key t must also be performed in the NTT domain, as the key itself is transmitted in that format. This design choice helps balance performance and implementation simplicity, while preserving Kyber's security and modularity.

Further advancing the exploration of high-performance PQC implementations, Gupta et al. [18] presented an optimized GPU-based implementation of Kyber that achieved remarkable performance gains.

When executed on an NVIDIA Quadro GV100 GPU, the implementation demonstrated a speedup of approximately 51.05× over the baseline C implementation, achieving an impressive throughput of 473,000 key exchanges per second. Even when compared to an optimized AVX2 version running on high-end CPUs, the GPU-based design retained a significant 14.6× performance advantage.

This study gracefully showed the potential of GPU acceleration for PQC algorithms, particularly for large-scale or high-throughput applications such as secure communication servers or IoT gateways requiring rapid key exchanges.

Parallel research has also investigated the benefits of hybrid cryptographic systems that combine classical and post-quantum algorithms.

One such proposal explored the integration of NTRU with Rijndael (AES) in a hybrid encryption framework using parallel multithreading [19].

The architecture suggested used multithreaded servers, which had the ability to run both algorithms simultaneously, which essentially spread the workload among the multiple CPU cores. This architecture took advantage of the natural parallelism of modern multicore architectures, and it attained significant gains in system throughput.

The experiment showed how classical symmetric encryption algorithms would be compatible with and work alongside post-quantum asymmetric algorithms to create a more resilient performance-optimized hybrid cryptographic framework which is a strategy that is specifically useful in a transitional cryptographic setting.

An ESP32 platform paper contained an optimized multicore version of Kyber to support resource-constrained IoT devices [13]. The authors also rearranged the Kyber computational tasks in order to utilize the dual-core design of the

ESP32 microcontroller. Through the independent execution of the components of the algorithm by multiple threads, including generation of keys, encapsulation, and decapsulation, the implementation demonstrated a quantifiable speedup of 1.21 x generation of keys, 1.22 x encapsulation and 1.20 x decapsulation.

Another counterintuitive observation that was made in the study is that parallelization does not necessarily lead to the enhancement of performance.

There was a slight slowdown in decryption process brought about by the low fraction of the code that can be parallelized and the overhead added by thread control, semaphore control and synchronization.

In the case of IND-CPA decryption, the computational cost of dual-core coordination exceeded the time cost-saving of a parallel execution thus resulting in a net slowdown.

This also points to a central problem in optimizing PQC in embedded systems that is the tradeoff between overheads due to parallelization and actual performance improvements. Future research can possibly avoid these problems by examining the various forms of parallelization techniques.

In 2024, a comprehensive survey of the computational requirements of post-quantum cryptography in Internet of Things (IoT) systems was published [20].

The authors have reviewed 35 recent articles on the topic of GPU parallelization methods of PQC schemes, FPGA, and developments on hardware and software acceleration. Their findings showed that PQC algorithms continue to pose a very complex computational and memory demand on lightweight IoT devices and this makes it challenging to practically implement their usage.

The survey underlined the fact that, though there has been a lot of progress in optimizing PQC of constrained devices, the current activities are still in a scattered fashion and do not possess a clear focus.

The authors came to the conclusion that more cooperation between software developers, hardware designers, and cryptographic standardization organizations should be the main focus of future research. Such interdisciplinary coordination is essential to ensure that the next generation of PQC solutions can be both secure and efficiently deployable across the landscape of IoT systems.

C. SECURITY AND STANDARDIZATION

A key cryptographic method that improves the security of public-key encryption (PKE) schemes is the Fujisaki-Okamoto (FO) transform [21].

It offers a general technique to upgrade a scheme that achieves full chosen-ciphertext attack (IND-CCA2) security from one that is only secure under chosen-plaintext attacks (IND-CPA).

To stop adversaries from taking advantage of decryption oracles, the FO transform combines cryptographic hashing and deterministic re-encryption. In simple terms, it ensures that any invalid ciphertext will yield a pseudorandom key

indistinguishable from a valid one, thereby preventing attackers from learning any information about the secret key through ciphertext manipulation or error observation.

Kyber achieves IND-CCA2 security by using this FO transform. Since Kyber's underlying encryption system which is derived from MLWE, is originally CPA-secure, the FO transformation allows Kyber to be safely deployed in real-world environments where active attackers may attempt to induce decryption failures or exploit oracle responses. This transformation is the main reason for Kyber's robustness and one of the primary basis for its selection in the NIST standardization process.

Decryption failures play an essential role in the FO-based construction and Kyber allows for these failures. When a ciphertext fails to decrypt properly, the implementation does not expose this event as a decryption error. Instead, it outputs a pseudorandom "fallback" key, which ensures that an adversary cannot distinguish whether a decryption succeeded or failed.

This mechanism effectively prevents the formation of decrypt-oracle-style attacks, where an attacker could otherwise use decryption success or failure as a side channel to infer information about the secret key. The presence of such controlled decryption failures is thus a carefully engineered defensive measure integral to Kyber's CCA security model and not a weakness.

The potential to take advantage of these decryption failures under specific quantum adversary models has been investigated theoretically.

It has been proposed that Grover's algorithm, which searches through parameter spaces more effectively than traditional brute force, could be used to increase the likelihood of decryption failure.

If Kyber's parameters are not carefully selected, such failure-boosting attacks could theoretically reveal some information about the secret key. But according to the Kyber specification document, these attacks only happen very infrequently, far below acceptable security thresholds for the selected parameter sets.

Although Kyber is widely regarded as one of the most efficient and well-studied post-quantum schemes, it has nonetheless been the subject of several notable cryptanalytic studies aimed at stress-testing its security assumptions.

One such study demonstrated a chosen-plaintext attack (CPA) that combined traditional lattice reduction techniques with optimized algebraic manipulation, managing to break a Kyber instance in approximately 16 minutes under certain experimental conditions [22].

In 2025, the researchers followed by a refined form of this approach as a Plantard algorithm, which was an improvement of the original method in terms of computational efficiency (almost 50 percent better) than the original method had been at that time [23].

Even though these attacks are on weakened parameter sets or simplified models, as opposed to the standardized

configurations, they are useful proof-of-concept tests, which further understand the resistance margins of Kyber and guide the process of parameter tuning in future applications.

A key mismatch attack which uses a quantum binary search strategy is another important development in cryptanalysis [24].

Within this approach, a pairwise-parallel mismatch detection scheme enables the attacker to test several ciphertext-key pairs at the same time with quantum parallelism, which significantly reduces how many oracle calls the attacker needs as well as the overall computation time. The paper claimed efficiency improvements of 61 per cent with Kyber512 and about 57 per cent with both Kyber768 and Kyber1024 compared to previous quantum-aided attacks.

Although the results are more of theoretical interest, they also point to the changing nature of quantum cryptanalysis with a focus on the further analysis of Kyber and other lattice-based schemes under quantum attack models.

Although Kyber has been well-grounded mathematically, side-channel vulnerabilities are a cause of concern.

Side-channel attacks (SCAs) take advantage of physical leakage, like timing, power usage or electromagnetic emissions, to deduce secret information in cryptographic computations. The latter was proven by a recent study [25] which showed that in implementations that use static keys, side-channel attacks which are carefully engineered and certain conditions are met, it is possible to extract secret keys of Kyber successfully. The results highlight that the implementation security is as important as theoretical security, especially in an embedded or IoT system where the physical access or proximity attack is possible.

These papers demonstrate correctly that although the Kyber FO-transformed KEM has a robust IND-CCA2 security in the standard model, ongoing cryptanalytic assessment is highly valuable to ensure the safety of this scheme in the long term.

Continuous discovery of theoretical, quantum, and side-channel attacks give useful feedback to the research community, and lead to the advancement of the cryptographic design as well as its practical applications. Notably, all the reported attacks have failed to undermine the normal parameters as stipulated by Kyber, rather, they provide tests of stress that confirm the maturity and stability of the basic assumptions of the scheme.

D. MULTICORE IMPLEMENTATION CONSIDERATIONS

Based on the literature review, this section identifies the key points to keep in mind.

1. Parallelism strategy: Prefer coarse-grained overlap of SHAKE/sampling and NTT/polyvec arithmetic across the two cores; expect the most gain in key generation and encapsulation, with limited benefit in decapsulation due to smaller parallelizable fractions and synchronization overheads.

2. FO transform and failure handling: Implement decapsulation checks and fallback key selection via constant-time masking (no branches, no early returns), and use constant-time comparisons for all validity checks.
3. Side-channel and fault robustness: Eliminate secret-dependent branches and memory access patterns, maintain uniform access strides in SRAM, zeroize secrets deterministically, and add lightweight redundancy checks to detect basic faults.
4. Concurrency control: Use minimal synchronization.

IV. METHODOLOGY

A. PLATFORM

Raspberry Pi Pico 2 W is a low-cost, Wi-Fi/Bluetooth microcontroller board built around Raspberry Pi's RP2350A, a dual-core microcontroller with selectable Arm Cortex-M33 or Hazard3 RISC-V cores running up to 150 MHz, paired with 520 KB on-chip SRAM and 4 MB QSPI flash on the board. The board integrates a CYW43439 radio for 2.4 GHz 802.11n Wi-Fi and Bluetooth 5.2, enabling compact wireless IoT designs without external modules. RP2350 adds a documented security architecture with Arm TrustZone for Cortex-M, signed boot, 8 KB antifuse OTP for keys, a hardware TRNG, and a SHA-256 accelerator, improving both performance and trust for cryptographic workloads compared to the prior RP2040 generation. For high-throughput I/O, RP2350 provides three PIO blocks (12 total state machines), a full crossbar bus fabric, and USB 1.1 device/host, which together offload and parallelize peripheral handling while the two CPU cores focus on application logic. The RP2350A package used by Pico 2 W exposes 30 GPIOs (with 4 optional ADC inputs), maintaining pinout compatibility with earlier Pico boards while delivering higher core clocks and more RAM for compute-bound tasks.

B. TOOLS

Arduino IDE (Arduino-Pico core): Pico 2 W is supported by Earle Philhower's Arduino-Pico core, which targets RP2350 via Pico SDK 2.x and exposes the full board family under "Raspberry Pi Pico/RP2040/RP2350" in Boards Manager for straightforward C/C++ development in Arduino IDE 2.x. The core supports multicore, SPI/I2C, USB (TinyUSB), file systems (LittleFS/SDFS), lwIP networking, and PSRAM features, and it allows selecting the CPU architecture (Arm Cortex-M33 or Hazard3 RISC-V) from Tools → CPU architecture when applicable. Typical setup involves adding the core's Boards Manager URL, installing the package, selecting the Pico 2 W board, and uploading via BOOTSEL UF2 workflow or serial, mirroring Pico-family workflows with added RP2350 capabilities.

Visual Studio Code + Pico VS Code extension: The official Raspberry Pi Pico VS Code extension streamlines creating, building, running, and debugging C/C++ projects using the Pico SDK, with integrated hardware API

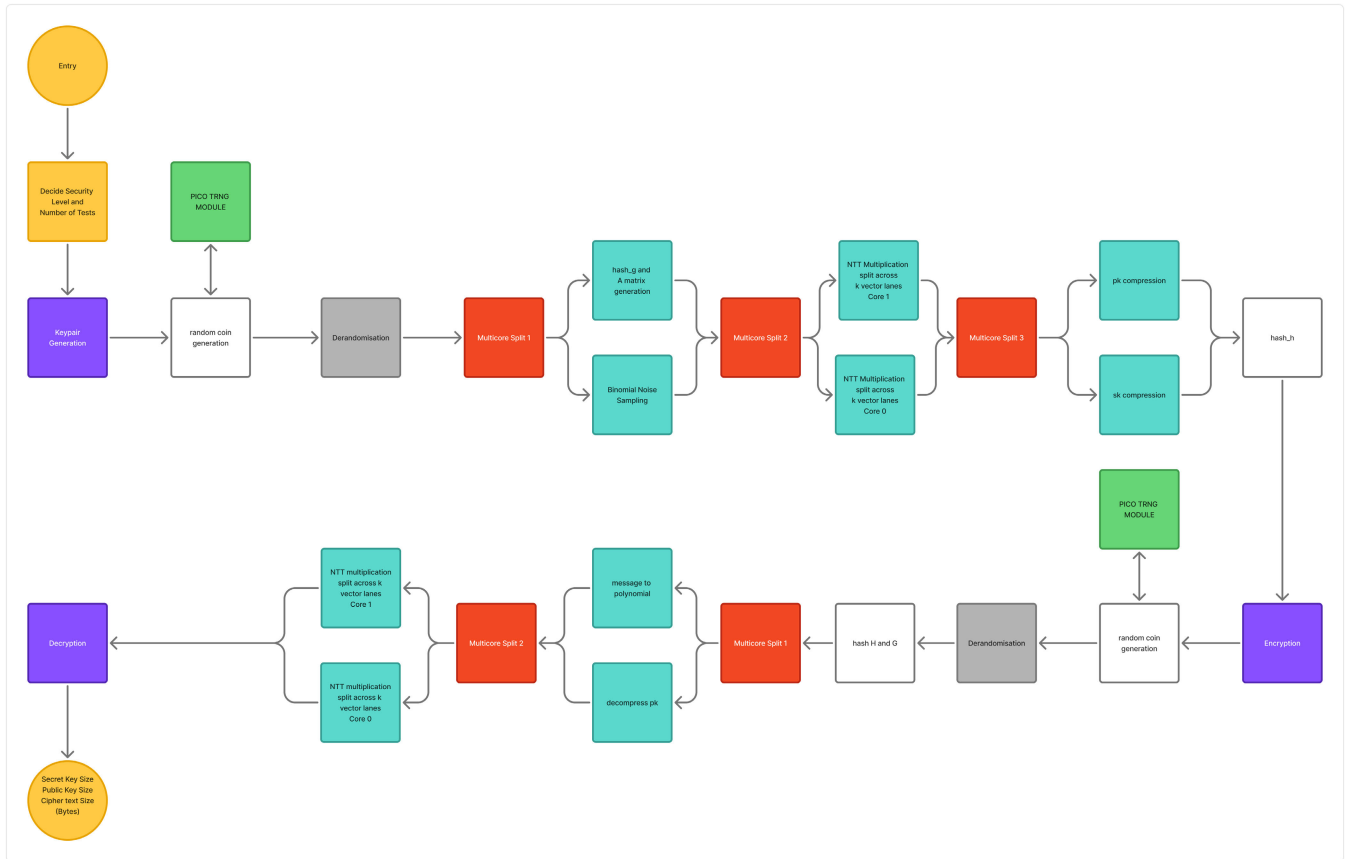


FIGURE 4. Flowchart depicting changes made for the multicore implementation. The boxes colored with cyan depict the core split.

documentation, example project templates, and one-click tasks tailored for Pico, Pico W, and Pico 2 boards. Installation is via the VS Code Marketplace (or OpenVSX) and provides a dedicated Pico sidebar for new projects (including board selection like Pico 2/Pico 2 W), build/flash actions, and debugging support, reducing manual SDK/CMake configuration for faster iteration. The extension's integrated documentation and project scaffolding make it well-suited for mixed compute/I/O workloads that leverage RP2350's dual cores and PIO, while remaining aligned with upstream Pico SDK practices for portability and long-term maintainability.

The implementation was developed using the Raspberry Pi Pico SDK v2.1.1. The specific configuration details are as follows:

- **Compiler:** The project was cross-compiled using the ARM GNU Toolchain (14.2.Rel1) with the arm-none-eabi-gcc compiler version 14.2.1.
- **Optimization:** All modules were compiled with the '-O3' optimization flag to maximize execution speed.
- **MCU Frequency:** The RP2350A was clocked at its native default frequency of 150 MHz.
- **System Environment:** The code executed in a bare-metal environment to eliminate the non-deterministic scheduling overhead typical of Real-Time Operating Systems (RTOS). **Background Services:** To isolate the energy

cost of the Kyber kernels, background services were minimized:

- The CYW43 wireless architecture was set to `pico_cyw43_arch_none` to prevent radio-frequency (RF) power spikes.
- Debug logging via USB/UART (`CYW43_LOG_ENABLED`, `PICO_CYW43_ARCH_DEBUG_ENABLED`) was explicitly disabled to ensure zero I/O-related power overhead during the active measurement window.

C. IMPLEMENTATION

The official implementation of the Kyber key encapsulation mechanism was obtained from the PQ-Crystals repository, which hosts the reference implementations used in the NIST Post-Quantum Cryptography (PQC) standardization process. Building upon this foundation, a novel multicore implementation and a Raspberry Pi Pico port were developed to evaluate the feasibility and efficiency of parallelized post-quantum cryptography on resource-constrained embedded platforms.

The initial phase of development involved a systematic examination of the Kyber algorithm to identify potential avenues for fine-grained and coarse-grained parallelization.

Fine-grained parallelism is carried out through low-level operations, such as element-wise arithmetic or loop unrolling, that can execute concurrently within a single function, whereas coarse-grained parallelism refers to dividing the algorithm into larger, functionally distinct components suitable for concurrent execution.

After extensive profiling and iterative testing, coarse-grained task partitioning was determined to be the most effective approach, as it minimized synchronization overhead and inter-core communication. These are two critical factors in the context of low-power embedded systems.

The multicore implementation follows a static fork-join execution model. At predefined split points, computationally intensive Kyber operations such as matrix generation, noise sampling, NTT multiplication, and key compression are partitioned across the two cores. Work is assigned deterministically, either by dividing independent functional blocks or splitting polynomial/vector computations across lanes.

Synchronization was achieved using inter-core message passing (mailbox/FIFO signaling) combined with blocking waits, forming barrier points at the end of each parallel region. A stage does not proceed until both cores complete their assigned tasks and signal readiness.

The scheduling behavior is static and phase-oriented. Execution alternates between sequential and parallel sections with explicit join points. This minimizes scheduling overhead, ensures deterministic timing, and is well-suited for structured cryptographic workloads on dual-core microcontrollers.

Inter-core coordination is implemented using a message box style signaling model combined with blocking synchronization. At each multicore split point, Core 0 assigns a predefined task to Core 1 and signals execution using a shared notification mechanism. Both cores execute their respective workloads independently and synchronize at a barrier, where progression to the next stage occurs only after both tasks have completed. This fork-join execution model ensures deterministic behavior and avoids race conditions.

Task granularity is defined at the level of algorithmic stages and polynomial/vector operations. Computation heavy components such as matrix generation, noise sampling, NTT-related processing, and compression steps are partitioned into independent sub-tasks and distributed across cores. Intermediate results are exchanged through shared memory buffers, where each core writes to designated regions to prevent overwrite conflicts. Core 1 remains idle during sequential regions, is activated at each split point, executes its assigned computation, and then returns to a waiting state after synchronization.

The key generation phase was restructured to exploit multicore execution while maintaining algorithmic correctness and determinism. The hash function and A-matrix generation operations, which are both computationally intensive and largely independent, were assigned to core 1, whereas noise sampling was executed on core 0. Following these steps, the polynomial multiplications within the Number Theoretic Transform (NTT) domain were divided between the two

cores, allowing simultaneous execution of high-cost arithmetic operations. Upon completion, public key packing was delegated to core 1, and secret key packing was carried out on core 0. This task distribution effectively balanced computational load while minimizing data dependency stalls.

A comparable parallelization scheme was applied to the encryption phase. Core 1 was tasked with transforming the message byte array into its polynomial representation, while core 0 concurrently decompressed the public key (pk). The NTT-domain multiplications, which constitute one of the most computationally demanding steps in Kyber, were again split between the two cores to maximize throughput. This division of labor resulted in observable reductions in execution time, particularly in sections involving polynomial arithmetic and modular transformations.

Conversely, the decryption phase was deliberately executed in a single-threaded manner. Both literature review and empirical evaluation confirmed that attempts to parallelize decryption yielded increased runtime and synchronization overhead without tangible benefits. The inherently sequential nature of certain decryption steps, especially those dependent on intermediate outputs, renders parallel execution inefficient. Consequently, decryption remained single-core to ensure optimal performance and maintain algorithmic integrity.

Beyond reducing runtime, the multicore implementation sought to maintain energy efficiency, as the activation of the second core introduces potential power overhead. To address this, a message-box model was implemented for inter-core communication. In this model, core 0 functions as the primary controller, orchestrating computation by dispatching both data and executable code segments to core 1 via shared memory buffers. Core 1 is activated only when required to perform low-to-moderate intensity tasks, thereby ensuring that its operational duration and consequently its power consumption, remains minimal. This design achieved a balance

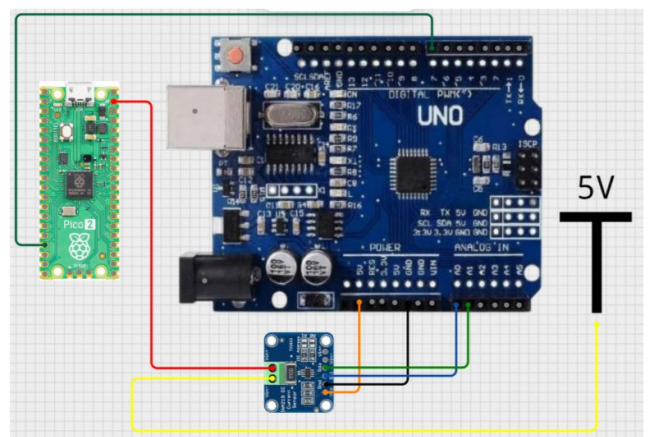


FIGURE 5. Circuit Diagram for monitoring power draw and runtime performance.

between performance enhancement and energy conservation, yielding a net speed-up with negligible power penalty.

A flowchart summarizing the changes can be seen in figure 4.

During implementation, a significant challenge arose from the 4 KB stack limitation per core on the RP2350 micro-controller. Several large data structures, particularly vector polynomial arrays, exceeded available stack space, causing program instability and runtime failures. To mitigate this issue, these variables were declared as static, relocating them from the stack to the global data segment. While effective in resolving memory constraints, this modification introduced a potential security risk.

In cryptographic applications, static variables retain sensitive intermediate states, such as secret coefficients or partial hash values, across function calls, exposing them to side channel analysis, memory inspection, or disclosure attacks. Following best practices from CERT Secure Coding and research on volatile memory protection, all static variables are explicitly zeroized post-use via compiler-proof methods (e.g., SecureZeroMemory or volatile memset), overwriting with zeros before function termination to eliminate remnants and uphold confidentiality [26], [27].

Following this, all sensitive data in static variables (e.g., secret coefficients, polynomials) is zeroized using a compiler-resistant volatile pointer loop after use:

```
void secure_zero ( void* v, size_t n)
{volatile uint8_t* p =
(volatile uint8_t *)v;
while (n--) *p++ = 0;
}
```

Our implementation uses RP2350's multicore FIFO message-box (mailbox) for inter-core synchronization, avoiding general shared memory while Core 0's pk[] output is read-only accessible by Core 1 during pack_pk. Potential side-channel risks include timing variations from FIFO contention and cache/PIPT attacks on polynomial temporaries. We mitigate these via: (1) Fixed-time NTT/Keccak primitives from the reference Kyber implementation (constant time across cores); (2) Deterministic scheduling where Core 0 (master) barriers Core 1 via multicore_fifo_pop_blocking() before shared reads; (3) Stack isolation with separate per core stacks (128KB SRAM partition); and (4) No sensitive shared state, secret polynomials remain core-local. These measures ensure execution time remains data-independent, with measured standard deviation <1% across 1000 and 100 multicore runs, matching single-core variance.

Our multicore Kyber implementation inherits constant-time properties from the PQ-Crystals reference Kyber design, which employs fixed-time table lookups, branchless arithmetic (e.g., conditional swaps via masks), and uniform-time NTT/Keccak-f primitives across all security levels (Kyber512/768/1024). Core-specific operations maintain data-independent execution: Core 0's polynomial sampling/NTT_even uses identical cycle counts regardless of coefficient magnitude; Core 1's NTT_odd follows identical paths. Multicore synchronization via

deterministic RP2350 FIFO barriers ensures no data-dependent stalls. These principles follow established guidelines [17], [28], [29].

Beyond theoretical constant-time properties, our multicore design eliminates risks through code-level measures, such as the secure zeroize function, which zeroizes all static variables after their usability has been exhausted. Secrets are kept locally within a single core and not transferred. Noise et al [256] remain in Core 0 SRAM (0 × 20040000.0 × 20047FFF stack partition); Core 1 accesses only read-only pk[] after multicore_fifo_push_blocking (sync_token) barrier. Timing is also uniform as branchless poly_compress() and deterministic FIFO yields <1%.

This work evaluates a single deterministic partitioning strategy designed to minimize coordination overhead and maintain predictable execution. The objective is to isolate and measure the performance impact of multicore execution rather than compare multiple partitioning strategies. Exploration of alternative splitting approaches is identified as future work.

To isolate the impact of multicore execution, the single-core baseline was measured using the same implementation with parallel execution disabled. This ensures that both configurations are identical in structure and logic, differing only in the use of the second core. As a result, the observed performance differences can be attributed directly to multicore task parallelism.

D. TEST BED

As seen in Figure 5, to measure the power consumption of the Kyber operations on the RP2350A, an INA219 current/power monitor was interfaced with an Arduino controller. The INA219 was configured using a 0.1 ohm shunt resistor with a 32 V/2 A calibration setting. The internal Analog-to-Digital Converter (ADC) was set to 12-bit resolution with a single sample conversion time of 532 μs. The Current LSB was 0.1 mA, providing a high-precision measurement of the current fluctuations during the KeyGen, Encapsulation, and Decapsulation phases. Power data was polled every 100 ms via the I2C interface.

The board was powered via USB at 5 V. No significant voltage droop was observed during cryptographic execution within the resolution of the measurement setup.

For each Kyber primitive, the firmware gates measurement windows by resetting the INA219's averaging/convert sequence and sampling current, voltage, and power at a fixed rate while the operation executes; cumulative energy is integrated as

$$E = \sum P_i \Delta t \quad (1)$$

over the window to obtain joules per operation. Where P_i denotes the instantaneous power measured at the i^{th} sampling interval, Δt is the sampling period (Execution time). Execution time is captured via on-chip cycle counters around the

TABLE 7. Mean runtimes for NTEST = 100 on Pico 2.

NTEST = 100	Kyber	Keygen (μ s)	Enc (μ s)	Dec (μ s)
Single-core	512	6045.61 $\pm$ 52.97	7071.97 $\pm$ 32.16	8678.60 $\pm$ 32.52
Single-core	768	9913.11 $\pm$ 53.71	11460.85 $\pm$ 39.99	13624.17 $\pm$ 47.62
Single-core	1024	15404.24 $\pm$ 69.76	17275.37 $\pm$ 62.42	20045.91 $\pm$ 65.65
Multicore	512	4381.21 $\pm$ 66.04	6814 $\pm$ 59.17	8398 $\pm$ 67.54
Multicore	768	7351.51 $\pm$ 72.24	11148.97 $\pm$ 58.06	13440.85 $\pm$ 73.53
Multicore	1024	11426.45 $\pm$ 82.78	16927.55 $\pm$ 66.10	19870 $\pm$ 82.81

TABLE 8. Mean runtimes for NTEST = 1000 on Pico 2.

NTEST = 1000	Kyber	Keygen (μ s)	Enc (μ s)	Dec (μ s)
Single-core	512	6032.53 $\pm$ 29.49	7063.82 $\pm$ 27.13	8662.27 $\pm$ 26.88
Single-core	768	9897.59 $\pm$ 37.88	11451.46 $\pm$ 36.57	13607.80 $\pm$ 37.68
Single-core	1024	15386.53 $\pm$ 48.63	17255.62 $\pm$ 47.47	20017.10 $\pm$ 48.65
Multicore	512	4344.69 $\pm$ 28.38	7137.93 $\pm$ 32.63	8835.34 $\pm$ 34.14
Multicore	768	7313.26 $\pm$ 42.66	11118.12 $\pm$ 41.22	13402.10 $\pm$ 43.63
Multicore	1024	11390.88 $\pm$ 52.28	16893.13 $\pm$ 51.13	19832.18 $\pm$ 53.10

target code segment; runtime in milliseconds is computed as

$$t = \frac{C}{f_{\text{cpu}}} \quad (2)$$

where C demotes the total number of clock cycles required to complete the operation and f_{cpu} is the processor frequency expressed in cycles per second (Hz).

V. RESULTS

This section reports runtime microbenchmarks for Kyber-512, Kyber-768, and Kyber-1024 key generation, encapsulation, and decapsulation on the Raspberry Pi Pico 2 W tabulated in Tables 7 and 8. It compares a baseline single-core implementation against a multicore variant under two sampling regimes (NTEST = 100 and NTEST = 1000). It also underlines the current draw of the Pico 2 W during Kyber execution under single-core and multicore configurations.

A. SETUP AND METRICS

We measure average runtime in microseconds (μ s) for key generation (keygen), encapsulation (enc), and decapsulation (dec), and report the corresponding multicore speedups as the ratio of single-core over multicore mean times for each operation and parameter set.

B. SINGLE-CORE BASELINE

For NTEST = 100, single-core mean runtimes (in μ s) are: Kyber-512 (keygen 6045.61, enc 7071.97, dec 8678.60), Kyber-768 (keygen 9913.11, enc 11460.85, dec 13624.17), and Kyber-1024 (keygen 15404.24, enc 17275.37, dec 20045.91). A clear monotonic increase with security level is observed across all three operations.

For NTEST = 1000, single-core mean runtimes remain closely aligned with the NTEST=100 measurements: Kyber

512 (keygen 6032.53, enc 7063.82, dec 8662.27), Kyber 768 (keygen 9897.59, enc 11451.46, dec 13607.80), and Kyber-1024 (keygen 15386.53, enc 17255.62, dec 20017.10). The deviations are below 0.3% across all parameter sets, indicating strong measurement stability and negligible drift with increased trials.

C. MULTICORE PERFORMANCE

For NTEST = 100, multicore mean runtimes (in μ s) are: Kyber-512 (keygen 4381.21, enc 6814.00, dec 8398.00) and Kyber-768 (keygen 7351.51, enc 11148.97, dec 13440.85). Consistent reductions are observed relative to the single-core baseline, particularly for key generation.

For NTEST = 1000, multicore mean runtimes are: Kyber 512 (keygen 4344.69, enc 7137.93, dec 8835.34), Kyber-768 (keygen 7313.26, enc 11118.12, dec 13402.10), and Kyber 1024 (keygen 11390.88, enc 16893.13, dec 19832.18). Key generation benefits most strongly from parallelization, while encapsulation and decapsulation exhibit smaller but still measurable reductions. The close agreement between NTEST = 100 and 1000 confirms the reproducibility of the multicore scheduler and work partitioning.

D. SPEEDUPS

Speedups are computed as fractional reductions relative to single-core mean runtimes.

For NTEST = 100:

Kyber-512: keygen 27.5%, enc 3.6%, dec 3.2%

Kyber-768: keygen 25.8%, enc 2.7%, dec 1.3%

For NTEST = 1000:

Kyber-512: keygen 28.0%, enc-1.1% (slight slowdown), dec-2.0% (slight slowdown)

Kyber-768: keygen 26.1%, enc 2.9%, dec 1.5%

Kyber-1024: keygen 26.0%, enc 2.1%, dec 0.9%

Overall, multicore parallelization consistently accelerates key generation by approximately 26-28%, while encapsulation and decapsulation show modest improvements (typically 1-4%), with minor variance across sampling sizes. The larger gains in key generation align with the higher proportion of parallelizable workload in matrix generation and NTT-heavy phases.

All timings in Tables 7 and 8 represent mean $\pm \sigma$ over $N=100$ and $N=1000$ independent executions using monotonic time_us_64() counters.

Typical standard deviation observed: $<1\%$ CV (coefficient of variation), confirming result reproducibility.

Runtime measurements were averaged over 100 and 1000 executions, and statistical variability was quantified using standard deviation, coefficient of variation (CV), and 95% confidence intervals. For the single-core implementation, the CV remained below 1% across all operations, indicating highly stable measurements. In the multicore configuration, variability increased slightly due to synchronization and shared resource effects but remained low ($CV \leq 1\%$). The observed reduction in key generation time ($\sim 27\text{-}28\%$) is substantially larger than the measured run-to-run variation and confidence bounds, confirming that the improvement represents a true performance gain rather than measurement noise. Changes observed in encapsulation and decapsulation were comparatively small (within 1-2%) and closer to the noise floor, suggesting limited parallelization benefits for these stages on the evaluated platform.

1) SPEED-UP DEFINITION

We report the speedup as a percentage reduction in runtime, not as a multiplicative ratio. This paragraph defines our speed up percentage. Formally,

$$\text{speedup_pct} = \frac{T_{\text{single}} - T_{\text{multi}}}{T_{\text{single}}} \times 100 \quad (3)$$

where T_{single} is the runtime taken by the single-core version and T_{multi} is the time taken by the multicore version.

For clarity, the multiplicative speedup factor is

$$\text{speedup_factor} = \frac{T_{\text{single}}}{T_{\text{multi}}} \quad (4)$$

And these two are related by

$$\text{speedup_factor} = \frac{1}{1 - \text{speedup_pct}/100} \quad (5)$$

E. FINE-GRAINED PROFILING TESTS

We instrumented the single-core and multicore implementations to record stage-level wall-clock timings for the following components:

- Matrix generation (hash/XOF expansion)
- Noise sampling
- NTT and inverse NTT
- Polynomial/vector multiplication
- Compression/packing and reduction
- Message conversion/unpacking

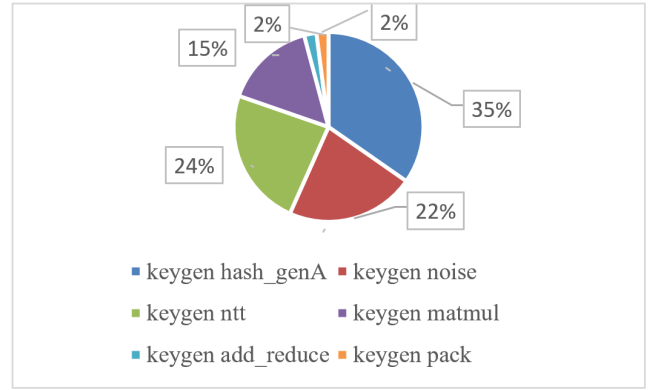


FIGURE 6. Fine-grained profiling results for Kyber512 key generation.

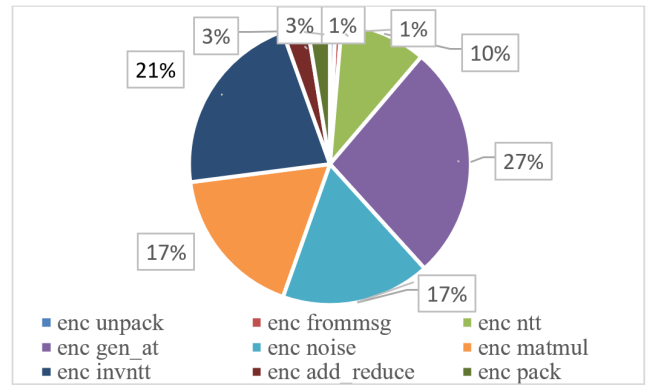


FIGURE 7. Fine-grained profiling results for Kyber512 encryption.

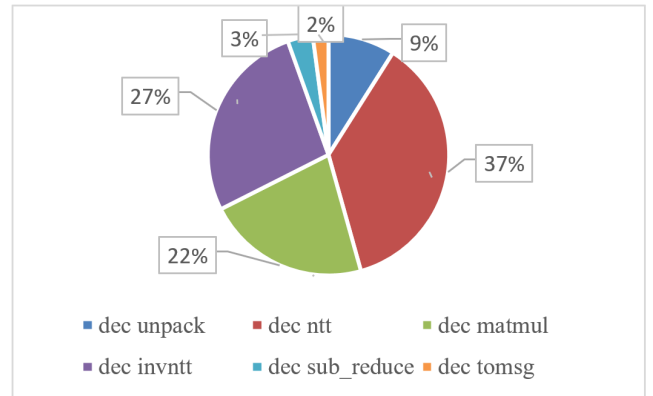


FIGURE 8. Fine-grained profiling results for Kyber512 decryption.

Measurements were collected over 1000 independent runs, and the timings were normalized to obtain percentage contributions within each phase (key generation, encapsulation, and decapsulation).

As can be seen in Figures 6, 7, and 8 results for single-core clearly highlight the dominant computational kernels:

- Key generation: dominated by hash/XOF expansion (35%), NTT (24%), noise sampling (22%), and matrix multiplication (15%).

TABLE 9. Flash and static RAM footprint (Kyber512).

Metric	Single-Core	Multicore	Notes
Flash (.text + .rodata + metadata)	≈ 281 KB	≈ 292 KB	~3-4 KB increase due to worker code & FIFO logic
Static RAM (.data + .bss + vectors)	≈ 9.7 KB	≈ 36.5 KB	Larger working buffers moved to .bss for safe sharing
Heap (reserved)	2 KB	2 KB	No dynamic allocation observed

TABLE 10. Runtime stack usage (Measured).

Configuration	Core0 Stack	Core1 Stack	Remarks
Single-Core	3,312 B	-	Within Pico 4 KB stack limit
Multicore	1,048 B	4,128 B	Stack pressure shifts to worker core

TABLE 11. Total RAM footprint (Estimated Runtime Requirement).

Configuration	Static RAM	Stack (total)	Heap	Total RAM
Single-Core	9.7 KB	3.3 KB	2 KB	≈ 15 KB
Multicore	36.5 KB	5.2 KB (1.0 + 4.1)	2 KB	≈ 44 KB

TABLE 12. Key memory Trade-offs.

Aspect	Single-Core	Multicore
Flash overhead	Lower	Slightly higher (+~4 KB)
Static RAM	Very low	Higher (shared working buffers)
Stack pressure	Concentrated on one core	Distributed across cores
Heap usage	None	None
Fit within RP2350 SRAM (520 KB)	Large margin	Large margin

- Encapsulation: dominated by matrix generation (27%), inverse NTT (21%), noise sampling (17%), and multiplication (17%), with NTT contributing 10%.
- Decapsulation: dominated by NTT (37%), inverse NTT (27%), and multiplication (22%), with smaller contributions from unpacking and reductions.

These results provide a precise breakdown of where computation time is spent, and help identify the most impactful targets for optimization and parallelization.

To further explain the observed performance differences, we analyzed the multicore execution profile at the component level. During key generation, the workload is effectively split across cores, with Core 0 primarily handling noise sampling (1245 μ s) and NTT (1278 μ s), while Core 1 executes a substantial portion of matrix generation (1948 μ s) and polynomial multiplication (445 μ s). This overlap allows compute-intensive stages to run concurrently, leading to the strongest acceleration in the setup phase.

In encapsulation, the dominant costs remain matrix generation (3303 μ s), inverse NTT (2649 μ s), and noise sampling (2085 μ s), mostly executed on Core 0, while Core 1 handles matmul (791 μ s) and message conversion (100 μ s). The partial concurrency across cores results in moderate overall speedup.

Decapsulation, in contrast, is dominated by a shorter chain of dependent operations: NTT (599 μ s), inverse NTT (441 μ s), and matmul (358 μ s), with limited opportunity for overlapping execution, similar to the single-core implementation.

Overall, these measurements confirm that the observed speedups are strongly aligned with the fraction of execution time spent in parallelizable arithmetic kernels and the degree to which their execution can be overlapped across cores.

F. MEMORY USAGE ANALYSIS

Memory usage was evaluated through ELF inspection and runtime stack instrumentation.

For the single-core Kyber512 implementation, the firmware occupies approximately 281 KB of flash and 9.7 KB of static RAM, as can be seen in Table 9. Runtime measurements show a peak stack usage of 3.3 KB, well within the 4 KB stack limit of the RP2350 (Table 10). No dynamic memory allocation was observed.

In the multicore implementation, flash usage increases marginally to approximately 292 KB. Static RAM rises to 36.5 KB due to the relocation of large working buffers into global static memory to enable safe sharing across cores. Runtime measurements show peak stack usage of 1.0 KB on Core0 and 4.1 KB on Core1, indicating that stack pressure is redistributed to the worker core while remaining within device limits.

The single-core version operates within a 4 KB stack budget, while the multicore version distributes stack usage across cores, with the worker core approaching but not exceeding typical embedded stack constraints.

Overall, the multicore design trades increased static RAM usage for reduced per-core stack pressure and improved performance, while remaining well within the RP2350's 520 KB SRAM capacity (Table 11).

Table 12 highlights the key memory trade-offs between the single-core and multicore implementations. The single-core configuration exhibits lower flash overhead and very low static RAM usage, with stack consumption concentrated on a single core. In contrast, the multicore design introduces a modest flash increase (approximately +4 KB) and higher static RAM usage due to shared working buffers, while distributing stack pressure across both cores. Neither configuration requires heap allocation. Importantly, both designs operate well within the 520 KB SRAM constraint of the RP2350, leaving a substantial memory margin despite the increased resource usage in the multicore variant.

TABLE 13. Energy per operation for single-core Kyber.

Version (Singlecore)	Phase	EnergyPerOp (mJ/op)
512	Keygen	0.1256
512	Encapsulation	0.1456
512	Decapsulation	0.1784
768	Keygen	0.2090
768	Encapsulation	0.2398
768	Decapsulation	0.2864
1024	Keygen	0.3232
1024	Encapsulation	0.3608
1024	Decapsulation	0.4228

TABLE 14. Energy per operation for multicore Kyber.

Version (Multicore)	Phase	EnergyPerOp (mJ/op)
512	Keygen	0.1006
512	Encapsulation	0.1530
512	Decapsulation	0.1870
768	Keygen	0.1652
768	Encapsulation	0.2332
768	Decapsulation	0.2846
1024	Keygen	0.2606
1024	Encapsulation	0.3586
1024	Decapsulation	0.4206

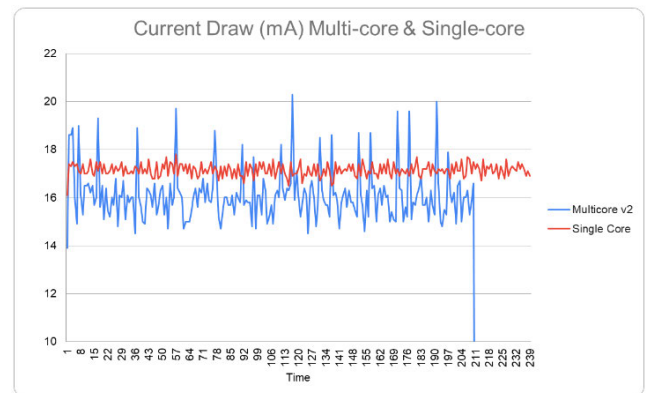
G. POWER DRAW TESTS

To complement runtime microbenchmarks, we measured the current draw of the Pico 2 W during Kyber execution under single-core and multicore configurations. Two test suites were executed; each averaged over 1000 independent runs to mitigate run-to-run variation.

Peak current values are constrained by the INA219 sampling rate and may under-report short transient spikes. Therefore, peak-current observations should be interpreted as approximate and limited to observable resolution.

1) AGGREGATE POWER CONSUMPTION

The first suite measured overall power consumption using a 10ms polling interval. This provided both total energy usage and definitive runtime, cross-validated against the Pico's internal clock. Tables 13 and 14 summarize the measured energy per-operation for each Kyber parameter set and phase. For key generation, multicore reduced energy from 0.1256 to 0.1006mJ/op (Kyber-512), 0.2090 to 0.1652mJ/op (Kyber-768), and 0.3232 to 0.2606mJ/op (Kyber-1024). Encapsulation and decapsulation were less consistent: encapsulation increased slightly for Kyber-512 (0.1456 to 0.1530mJ/op) but decreased for Kyber-768 (0.2398 to 0.2332mJ/op) and Kyber-1024 (0.3608 to 0.3586mJ/op), while decapsulation increased for Kyber-512 (0.1784 to 0.1870mJ/op) yet decreased marginally for Kyber-768 (0.2864 to 0.2846mJ/op) and Kyber-1024 (0.4228 to 0.4206mJ/op). Overall, multicore execution reduced aggregate energy usage by 8.9%, 10.9%, and

**FIGURE 9.** Comparison of current draw between multicore and single-core implementations.

10.7% for Kyber-512, Kyber-768, and Kyber-1024, respectively, despite increased instantaneous draw. The trade-off was higher transient peaks, with maximum current rising by approximately $1.3\times$ compared to single-core execution.

2) HIGH-RESOLUTION CURRENT PROFILING

As seen in Figure 9, the second suite targeted fine-grained current profiling, polling as fast as the Arduino host could capture over 1000 iterations of Kyber-1024. Average current draw for the multicore run was 16.397mA over 1.174s, compared to 17.145mA over 1.354s for single-core. Thus, the multicore configuration exhibited both a lower current-time integral and shorter completion time for the same workload. These high-resolution observations align with the phase level energy figures in Tables 13 and 14, where Kyber 1024 key generation drops from 0.3232 to 0.2606mJ/op and the remaining phases show small changes (0.3608 to 0.3586mJ/op for encapsulation and 0.4228 to 0.4206mJ/op for decapsulation). However, peak current increased from 17.8mA (single-core) to 20.3mA (multicore), which may stress regulators or battery-powered systems even though the total charge per operation declined.

Since total energy is proportional to the integral of power over time, reductions in execution time generally translate into lower energy consumption when average current remains similar. In our measurements, multicore execution introduces higher instantaneous current peaks due to both cores operating concurrently, with peak current increasing by approximately $1.3\times$. However, the overall runtime decreases substantially, and the average current slightly declines, resulting in a lower total current-time integral and therefore reduced energy per operation. This demonstrates that shorter execution duration can offset increased instantaneous power. That said, this relationship may not always hold; if parallelization significantly increases sustained current draw, introduces synchronization overhead, or causes voltage regulation inefficiencies, total energy consumption may not decrease despite runtime improvements.

TABLE 15. Runtime ablation results for Kyber (NTEST=100).

NTEST = 100	Kyber	Keygen (μ s)	Enc (μ s)	Dec (μ s)
Single-core	512	6032.53 $\pm$ 29.49	7063.82 $\pm$ 27.13	8662.27 $\pm$ 26.88
Single-core	768	9897.59 $\pm$ 37.88	11451.46 $\pm$ 36.57	13607.80 $\pm$ 37.68
Single-core	1024	15386.53 $\pm$ 48.63	17255.62 $\pm$ 47.47	20017.10 $\pm$ 48.65
Multicore (No Zeroization)	512	41245.27 $\pm$ 60.57	6800.59 $\pm$ 49.83	8362.69 $\pm$ 62.56
Multicore (No Zeroization)	768	6920.00 $\pm$ 80.87	10584.97 $\pm$ 61.07	12692.09 $\pm$ 78.00
Multicore (No Zeroization)	1024	10753.56 $\pm$ 82.88	16071.78 $\pm$ 71.63	18768.80 $\pm$ 82.42
Multicore	512	4344.69 $\pm$ 28.38	7137.93 $\pm$ 32.63	8835.34 $\pm$ 34.14
Multicore	768	7313.26 $\pm$ 42.66	11118.12 $\pm$ 41.22	13402.10 $\pm$ 43.63
Multicore	1024	11390.88 $\pm$ 52.28	16893.13 $\pm$ 51.13	19832.18 $\pm$ 53.10

TABLE 16. Runtime ablation results for Kyber (NTEST=1000).

NTEST = 1000	Kyber	Keygen (μ s)	Enc (μ s)	Dec (μ s)
Single-core	512	6032.53 $\pm$ 29.49	7063.82 $\pm$ 27.13	8662.27 $\pm$ 26.88
Single-core	768	9897.59 $\pm$ 37.88	11451.46 $\pm$ 36.57	13607.80 $\pm$ 37.68
Single-core	1024	15386.53 $\pm$ 48.63	17255.62 $\pm$ 47.47	20017.10 $\pm$ 48.65
Multicore (No Zeroization)	512	4112.58 $\pm$ 26.59	6776.86 $\pm$ 26.78	8331.16 $\pm$ 29.53
Multicore (No Zeroization)	768	6880.34 $\pm$ 42.06	10550.25 $\pm$ 40.31	12650.43 $\pm$ 45.15
Multicore (No Zeroization)	1024	10712.83 $\pm$ 55.01	16039.06 $\pm$ 54.97	18734.09 $\pm$ 53.66
Multicore	512	4344.69 $\pm$ 28.38	7137.93 $\pm$ 32.63	8835.34 $\pm$ 34.14
Multicore	768	7313.26 $\pm$ 42.66	11118.12 $\pm$ 41.22	13402.10 $\pm$ 43.63
Multicore	1024	11390.88 $\pm$ 52.28	16893.13 $\pm$ 51.13	19832.18 $\pm$ 53.10

H. ABLATION ANALYSIS

To better understand the impact of our design choices, we conducted an ablation study across three configurations: (i) single-core baseline, (ii) multicore execution without state zeroization, and (iii) multicore execution with zeroization enabled. Results are reported for Kyber-512, Kyber-768, and Kyber-1024 (Table 15 and Table 16), with NTEST indicating the number of independent runs used to compute the mean runtime and standard deviation.

1) EFFECT OF MULTICORE PARALLELIZATION

Across both NTEST=100 and NTEST=1000, multicore execution consistently improves performance for key generation, while providing smaller but measurable gains for encapsulation and decapsulation. For NTEST=1000, key generation time drops from 6032.53 μ s to 4344.69 μ s for Kyber-512 (27.98% speedup), from 9897.59 μ s to 7313.26 μ s for Kyber-768 (26.11%), and from 15386.53 μ s to 11390.88 μ s for Kyber-1024 (25.97%). Encapsulation shows marginal improvement: 7063.82 μ s $\rightarrow$ 7137.93 μ s (Kyber-512, slight regression), 11451.46 μ s $\rightarrow$ 11118.12 μ s (2.91% faster for Kyber-768), and 17255.62 μ s $\rightarrow$ 16893.13 μ s (2.10% faster for Kyber-1024). Decapsulation remains close to the single-core baseline, with changes within 1-3% depending on the parameter set. Similar trends are observed for NTEST=100, indicating that the benefits are stable across different sample sizes.

2) IMPACT OF REMOVING ZEROIZATION

The “multicore (no zeroization)” configuration isolates the cost of memory clearing operations. For NTEST = 1000, the key generation time further reduces to 4112.58 μ s (Kyber 512), 6880.34 μ s (Kyber 768), and 10712.83 μ s (Kyber 1024). Compared to multicore with zeroization enabled, this corresponds to an additional improvement of 5.34%, 5.92%, and 5.95%, respectively. Encapsulation and decapsulation also benefit slightly: for example, Kyber-768 encapsulation improves from 11118.12 μ s to 10550.25 μ s (5.11%), and decapsulation from 13402.10 μ s to 12650.43 μ s (5.61%). These gains quantify the overhead introduced by secure memory clearing. Stability across NTEST values.

Comparing NTEST=100 and NTEST=1000, the mean run times remain highly consistent across all configurations. However, the larger sample size produces tighter standard deviations, particularly for the multicore (no zeroization) configuration. For instance, Kyber-512 key generation shows 41245.27 $\pm$ 60.57 μ s at NTEST=100 versus 4112.58 $\pm$ 26.59 μ s at NTEST=1000. Overall, increasing NTEST primarily improves measurement stability rather than shifting central tendencies.

3) KEY OBSERVATIONS

- Multicore parallelization provides the largest benefit to key generation ($\approx$ 26-28%), moderate gains for encapsulation ($\approx$ 2-5%), and minimal change for decapsulation.

- Zeroization introduces a measurable overhead of roughly 5-6% across operations, most prominently in key generation.
- Increasing NTEST from 100 to 1000 reduces variance and strengthens confidence in the reported averages without significantly changing mean performance.

VI. CONCLUSION

In all parameter selections, key generation using multiple cores on the Raspberry Pi Pico 2 W was running, on average, 30 percent faster than in the single-core implementation; encapsulation and decapsulation made smaller but more predictable improvements that increased with security level. These gains were consistent across the sampling regimes (NTEST = 100 and NTEST = 1000) and the multicore scheduling and work partitioning are therefore reproducible.

Experiments with power-draw showed that not only the runtime was reduced with multicore execution, but also the mean current consumption was reduced. Complete power consumption decreased by 8.9 percent, 10.9 percent and 10.7 percent when it comes to Kyber-512, Kyber-768 and Kyber-1024. The high-resolution profiling of Kyber-1024 further revealed that average current had reduced to 17.145-16.397mA and completion time had reduced to 1.354-1.174s providing an overall 17 percent net efficiency increase per operation. The trade-off was mostly an increase in instantaneous peaks (a 17.8mA to 20.3mA), which can challenge power delivery components despite a reduction in total energy consumption. These findings imply that multicore execution offers a favorable ratio to duty-cycled or battery-powered deployments, assuming that regulator headroom is available to transient spikes.

A. LIMITATIONS

Although the experimental data illustrate steady performance gains through multicore parallelization, there are a variety of extraneous variables that can cause variability in the reported runtime and energy values. To begin with, the electrical noise in the environment can influence the power readings, especially where a small current change in hardware with resources constraints needs to be measured. Small variations between trials may be due to minor changes in the stability of the supply, background interference by other connected peripherals, and sensitivity of the measurement probe.

Second, the power-measurement arrangement itself implies some uncertainty. The precision of instantaneous current and energy estimates can be constrained by factors like sampling resolution, sensor tolerance, or time lag between execution phases and recorded readings. Despite the repeated trials that were done to minimize the random error and increase the reliability, the stated values are still to be regarded as close approximations, but not absolute values.

Third, thermal variation can have an impact on the nature of execution over time. Localized heating may influence clock stability, leakage currents, and total power consumption as the microcontroller is used. Even small temperature variations

between operating cycles can cause tiny variations in timing and energy behavior, particularly at longer or computationally intensive operations.

A combination of such factors has no impact on the functional correctness of the implementation, but can cause some small differences in measured performance and energy outcomes. However, because all comparisons were done under the same conditions, using the same hardware and procedure, the comparative disparities between single-core and multicore setups are still reflective and not meaningless.

B. FUTURE WORK

There are many ways in which this work can be extended to consolidate both the performance analysis as well as the overall application of multicore acceleration to post-quantum cryptography on small-scale devices to a larger extent. To begin with, further experiments can use more precise power measurement methods in the future so as to get sharper energy profiles. Improved sampling hardware or finer-scale measurement time steps would enable phase-based attribution of energy, and it would be possible to gain a better insight into which parts of the algorithm are the most suitable to be parallelized and where energy waste is being incurred.

Second, although the current research concentrates on one post-quantum scheme, the multicore model of execution can be assessed on other PQC primitives. Application of the methodology to other lattice-based schemes, and other signature and key encapsulation schemes would assist in finding out the extent to which the performance benefits that have been observed are generalizable, and whether task splitting is more beneficial to different algorithmic structures.

Third, the adaptive multicore scheduling strategies may be studied in future works. The existing layout employs a fixed allocation of work to cores that is simple and predictable, but might not provide maximum utilization. Dynamic or workload-conscious scheduling may be investigated to enhance efficiency by balancing the computational load in accordance with the runtime behavior, complexity of the stage, or data quantity. This can also minimize the wastage of time in the synchronization points and improve the overall throughput.

Lastly, it is possible to consider incorporating more thermal and power-consciousness into the scheduling choice. For example, the selective execution of only computationally intensive stages to balance the advantages of performance, energy consumption, and thermal stability, especially in long-running or battery-operated systems.

Overall, the results of multicore Kyber on the Pico 2 W indicate that parallelization can be used to both enhance runtime and minimize power use, which has turned the traditional belief that faster execution must be more expensive in terms of power usage. This underscores multicore parallelism as a feasible way to achieve effective post-quantum cryptography on low-cost embedded systems.

COMPETING INTERESTS

The authors declare that they have no competing interests.

ETHICS DECLARATION

This study does not involve human participants, human data, or human tissue. No animals were used or affected in the course of this research.

ROLES

Achintya Nigam: Conceptualization, Writing—original draft, Writing—review & editing, Methodology, Formal Analysis, Investigation, Software, Resources. Avanish Gharat: Writing—original draft, Writing—review & editing, Methodology, Formal Analysis, Software. Mehul Bhushan: Writing—original draft, Writing—review & editing, Formal Analysis, Software, Resources. Islabudeen Mohamed Meerasha: Conceptualization, Writing—review & editing, Supervision, Validation.

DATA

The datasets and code supporting the findings of this study are publicly available as follows:

- **Code Repository:** The source code for the KyberMulticore implementation is available at <https://github.com/AchintyaNigam/KyberMulticore>.
- **Runtime Performance Data:** Runtime performance results, including execution times for key generation, encryption, and decryption operations across different Kyber parameter sets, are provided in the following Google Spreadsheet: <https://docs.google.com/spreadsheets/d/14iegRUicScuRMzEju1etJmDJEvHUip-ahfKQqjXWcw/edit?usp=sharing>.
- **Power Consumption Data:** Power consumption measurements, detailing voltage, current, and energy consumption over time, are documented in this Google Spreadsheet: https://docs.google.com/spreadsheets/d/1Ep_5Cd9jqoc7FiEpgdl_HTDUdqoi1ysH/edit?usp=sharing&ouid=101126406031532358218&rtopof=true&sd=true.

All the datasets (Runtime Performance and Power Consumption Data) are also available at IEEE data port: <https://doi.org/10.21227/e4zw-fr04>

These resources are freely accessible and can be utilized to replicate the experiments and analyses presented in this study.

REFERENCES

- [1] *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, FIPS PUB 203, National Institute of Standards and Technology, Gaithersburg, MD, USA, Aug. 2024.
- [2] S. Diop, B. O. Sané, N. Diarra, and M. Seck, “NTRU-LPR IND-CPA: A new ideal lattices-based scheme,” *IACR Cryptol. ePrint Archive*, vol. 1, pp. 1–20, Jan. 2018. [Online]. Available: <https://eprint.iacr.org/2018/109.pdf>
- [3] I. Tzinos, K. Limnitis, and N. Kolokotronis, “Evaluating the performance of post-quantum secure algorithms in the TLS protocol,” *J. Surveill., Secur. Saf.*, vol. 3, no. 3, pp. 101–127, Sep. 2022, doi: [10.20517/jss.2022.15](https://doi.org/10.20517/jss.2022.15).
- [4] G. Alagic, D. Apon, D. Cooper, Q. Dang, T. Dang, J. Kelsey, J. Lichtinger, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, D. Smith-Tone, and Y.-K. Liu, “Status report on the third round of the NIST post-quantum cryptography standardization process,” National Institute of Standards and Technology, Gaithersburg, MD, USA, Tech. Rep. NISTIR 8413-upd1, Jul. 2022, doi: [10.6028/NIST.IR.8413-upd1](https://doi.org/10.6028/NIST.IR.8413-upd1).
- [5] S. Wang, “Research on post-quantum cryptosystem based on IoT devices,” *J. Phys., Conf. Ser.*, vol. 1883, no. 1, Apr. 2021, Art. no. 012082, doi: [10.1088/1742-6596/1883/1/012082](https://doi.org/10.1088/1742-6596/1883/1/012082).
- [6] D. V. Bailey, D. Coffin, A. Elbirt, J. H. Silverman, and A. D. Woodbury, “NTRU in constrained devices,” in *Proc. Int. Workshop Cryptogr. Hardw. Embed. Syst.*, Paris, France, Sep. 2001, pp. 262–272, doi: [10.1007/3-540-44709-1_22](https://doi.org/10.1007/3-540-44709-1_22).
- [7] O. M. Guillen, T. Pöppelmann, J. M. B. Mera, E. F. Bongenaar, G. Sigl, and J. Sepulveda, “Towards post-quantum security for IoT endpoints with NTRU,” in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Lausanne, Switzerland, Mar. 2017, pp. 698–703, doi: [10.23919/DATE.2017.7927079](https://doi.org/10.23919/DATE.2017.7927079).
- [8] J. Senor, J. Portilla, and G. Mujica, “Analysis of the NTRU post-quantum cryptographic scheme in constrained IoT edge devices,” *IEEE Internet Things J.*, vol. 9, no. 19, pp. 18778–18790, Oct. 2022, doi: [10.1109/JIOT.2022.3162254](https://doi.org/10.1109/JIOT.2022.3162254).
- [9] P. Grassl, M. Hudler, and M. Koschuch, “Low-performance embedded Internet of Things devices and the need for hardware-accelerated post-quantum cryptography,” in *Proc. 9th Int. Conf. Internet Things, Big Data Secur.*, Rome, Italy, Jan. 2024, pp. 329–338, doi: [10.5220/0012736800003705](https://doi.org/10.5220/0012736800003705).
- [10] B. Halak, T. Gibson, M. Henley, C.-B. Botea, B. Heath, and S. Khan, “Evaluation of performance, energy, and computation costs of quantum-attack resilient encryption algorithms for embedded devices,” *IEEE Access*, vol. 12, pp. 8791–8805, 2024, doi: [10.1109/ACCESS.2024.3350775](https://doi.org/10.1109/ACCESS.2024.3350775).
- [11] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, “PQM4: Testing and benchmarking NIST PQC on ARM cortex-M4,” presented at the 2nd NIST PQC Standardization Conf., Santa Barbara, CA, USA, Aug. 2019. [Online]. Available: <https://eprint.iacr.org/2019/844>
- [12] (Mar. 2024). *Post-Quantum Kyber Benchmarks (ARM Cortex M4)*. [Online]. Available: <https://www.wolfssl.com/post-quantum-kyber-benchmarks-arm-cortex-m4/>
- [13] F. Segatz and M. Hafiz, “Efficient implementation of CRYSTALS-KYBER key encapsulation mechanism on ESP32,” 2025, *arXiv:2503.10207*.
- [14] D. T. Nguyen, “Optimized software implementations of CRYSTALS-KYBER, NTRU, and saber using NEON-based special instructions of ARMv8,” presented at the 5th NIST PQC Standardization Conf., Rockville, MD, USA, Apr. 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:235399824>
- [15] F. Farahmand, D. T. Nguyen, V. B. Dang, A. Ferozpur, and K. Gaj, “Software/hardware codesign of the post quantum cryptography algorithm NTRUEncrypt using high-level synthesis and register-transfer level design methodologies,” in *Proc. 29th Int. Conf. Field Program. Log. Appl. (FPL)*, Farah, Spain, Sep. 2019, pp. 225–231, doi: [10.1109/FPL.2019.00042](https://doi.org/10.1109/FPL.2019.00042).
- [16] Y. Xing and S. Li, “A compact hardware implementation of CCA-secure key exchange mechanism CRYSTALS-KYBER on FPGA,” *IACR Trans. Cryptograph. Hardw. Embed. Syst.*, vol. 2021, pp. 328–356, Feb. 2021, doi: [10.46586/tches.v2021.i2.328-356](https://doi.org/10.46586/tches.v2021.i2.328-356).
- [17] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-kyber project,” NIST Post-Quantum Cryptography Standardization Project, Round 3 Submission (Version 3.01), 2021.
- [18] N. Gupta, A. Jati, A. K. Chauhan, and A. Chattopadhyay, “PQC acceleration using GPUs: FrodoKEM, NewHope, and kyber,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 3, pp. 575–586, Mar. 2021, doi: [10.1109/TPDS.2020.3025691](https://doi.org/10.1109/TPDS.2020.3025691).
- [19] Deepali and N. Kakkar, “Hybrid parallel multithreading encryption,” *Int. J. Comput. Sci. Eng.*, vol. 3, no. 7, pp. 12–20, Nov. 2015. [Online]. Available: <https://ijcseonline.org/index.php/j/article/view/587>
- [20] T. Liu, G. Ramachandran, and R. Jurdak, “Post-quantum cryptography for Internet of Things: A survey on performance and optimization,” 2024, *arXiv:2401.17538*.
- [21] E. Fujisaki and T. Okamoto, “Secure integration of asymmetric and symmetric encryption schemes,” in *Proc. 19th Annu. Int. Cryptol. Conf. Adv. Cryptol. (CRYPTO)*, Santa Barbara, CA, USA, Dec. 1999, pp. 537–554, doi: [10.1007/3-540-48405-1_34](https://doi.org/10.1007/3-540-48405-1_34).
- [22] Y.-T. Kuo and A. Takayasu, “A lattice attack on CRYSTALS-Kyber with correlation power analysis,” in *Information Security and Cryptology—ICISC 2023*, H. Seo and S. Kim, Eds., Singapore: Springer, Mar. 2024, pp. 202–220, doi: [10.1007/978-981-97-1235-9_11](https://doi.org/10.1007/978-981-97-1235-9_11).

- [23] C. Ghiban and M. O. Choudary, "Improved correlation power analysis attack on the latest cortex M4 kyber implementation," *Cryptography*, vol. 9, no. 1, p. 19, Mar. 2025, doi: [10.3390/cryptography9010019](https://doi.org/10.3390/cryptography9010019).
- [24] Y. Wang and J. Liu, "Quantum pairwise-parallel mismatch attack on kyber," *Phys. Scripta*, vol. 99, no. 11, Oct. 2024, Art. no. 115112, doi: [10.1088/1402-4896/ad827a](https://doi.org/10.1088/1402-4896/ad827a).
- [25] L. R. J. Constantin-Sukul, R. Ø. Gammelgaard, A. N. Henriksen, and D. Aranha, "Key recovery on static kyber based on transient execution attacks," in *Proc. Microarchitecture Secur. Conf. (MSC)*, Jun. 2025, pp. 104–119, doi: [10.46586/uasc.2025.006](https://doi.org/10.46586/uasc.2025.006).
- [26] S. Malliaros, C. Ntantogian, and C. Xenakis, "Protecting sensitive information in the volatile memory from disclosure attacks," in *Proc. 11th Int. Conf. Availability, Rel. Secur. (ARES)*, Salzburg, Austria, Aug. 2016, pp. 687–693, doi: [10.1109/ARES.2016.75](https://doi.org/10.1109/ARES.2016.75).
- [27] S. Arzt, S. Nadi, K. Ali, E. Bodden, S. Erdweg, and M. Mezini, "Towards secure integration of cryptographic software," in *Proc. ACM Int. Symp. New Ideas, New Paradigms, Reflections Program. Softw.*, Pittsburgh, PA, USA, Oct. 2015, pp. 1–13, doi: [10.1145/2814228.2814231](https://doi.org/10.1145/2814228.2814231).
- [28] D. J. Bernstein, "Curve25519: New Diffie–Hellman speed records," in *Proc. 9th Int. Theory Practice Public-Key Cryptogr. (PKC)*, New York, NY, USA, 2006, pp. 207–228, doi: [10.1007/11745853_14](https://doi.org/10.1007/11745853_14).
- [29] R. P. McEvoy, M. Tunstall, C. C. Murphy, and W. P. Marnane, "Differential power analysis of HMAC based on SHA-2, and countermeasures," in *Proc. Revised Sel. Papers 4th Int. Workshop Inf. Secur. Theory Practices (WISTP)*, Crete, Greece, 2007, pp. 317–332, doi: [10.1007/978-3-540-77535-5_23](https://doi.org/10.1007/978-3-540-77535-5_23).



ACHINTYA NIGAM is currently pursuing the Bachelor of Technology (B.Tech.) degree in computer science and engineering with Vellore Institute of Technology, Vellore, India.

He has held positions as a Software Engineer Trainee at Alumnus Software (Remote) and a Junior Software Developer at Xlancr, this has helped him gain an increased understanding of security flaws and pitfalls that occur during the development applications. His research focuses on post quantum cryptography and privacy preserving-protocols. His professional interests include penetration testing and software development.

Mr. Nigam is an active student and has presented his research in Post Quantum Cryptography at the Blockchain Kaigi 25 held at IIT Bombay, India. He has also participated in several hackathons and won the best in track award for sustainability for the DEVSOC'25.



AVANISH GHARAT is currently pursuing the Bachelor of Technology (B.Tech.) degree in computer science and engineering with Vellore Institute of Technology (VIT), where he has consistently demonstrated exceptional academic performance.

He completed an internship at Phillip Capital, a banking firm in Mumbai, where he gained practical experience in the financial sector. His previous roles include various summer jobs that have enriched his understanding of the industry. His research interests encompass data science and cybersecurity, reflecting his commitment to staying at the forefront of technological advancements. He has presented his research in post quantum cryptography at the Blockchain Kaigi 25 held at IIT Bombay, India. He has also participated in several hackathons and won the best in track award for sustainability for the DEVSOC'25.



MEHUL BHUSHAN is currently pursuing the Bachelor of Technology (B.Tech.) degree in computer science and engineering with Vellore Institute of Technology, Vellore, India. His research focuses on post quantum cryptography and optimizing algorithms. He has participated in many CTFs and is one of the Top 6% in the Tryhackme Community. He has a passion for offensive security and has completed the PNPT certification which involves practical exploitation of networks

involving OSINT, Enumeration and Active Directory Exploitation which has given him a profound understanding of the different points of vulnerabilities.



ISLABUDEEN MOHAMED MEERASHA received the B.E. degree in computer science and engineering from Madurai Kamaraj University, Madurai, India, in 2001, and the M.E. degree in computer science and engineering and the Ph.D. degree in information and communication engineering from Anna University, Chennai, India, in 2008 and 2021, respectively. He is currently an Assistant Professor (Senior Grade 2) with the School of Computer Science and Engineering

(SCOPE), Vellore Institute of Technology (Deemed to be University), Vellore, Tamil Nadu, India. He has more than 24 years of academic experience. He has more than 35 publications in refereed journals and reputed international conferences. His current research interests include cryptography and network security, blockchain, post quantum cryptography, and machine learning. He is a Life Member of the Computer Society of India (CSI) and Indian Society for Technical Education (ISTE). He has served as an active reviewer and the chair for many reputed international conferences and journals, including IEEE, Elsevier, and Springer.

...